2.7. Complexitatea operaţiilor

2.7.1. Ordine de mărime

Analiza eficienţei algoritmilor nu este întotdeauna un lucru simplu, fiind adesea foarte dificil de determinat timpul exact de execuţie al unui algoritm. În astfel de situaţii suntem forţaţi în determinarea unei aproximări a timpului de execuţie şi, frecvent, putem determina doar aproximări asimptotice.

În această secţiune vom prezenta principalele *ordine de mărime* prin intermediul cărora vom putea discuta despre comportarea asimptotică a algoritmilor. Reamintim că prin $\mathbf{R}_+$ ($\mathbf{R}_+^*$) s-a notat mulţimea numerelor reale pozitive (strict pozitive).

Fie g o funcţie de la $\mathbf{N}$ la $\mathbf{R}_+$. Considerăm următoarele mulţimi:

$$\begin{aligned}
\mathcal{O}(g) &= \{f : \mathbf{N} \to \mathbf{R}_+ | (\exists c \in \mathbf{R}_+^*)(\exists n_0 \in \mathbf{N})(\forall n \geq n_0)(f(n) \leq cg(n))\} \\
\Omega(g) &= \{f : \mathbf{N} \to \mathbf{R}_+ | (\exists c \in \mathbf{R}_+^*)(\exists n_0 \in \mathbf{N})(\forall n \geq n_0)(cg(n) \leq f(n))\} \\
\Theta(g) &= \{f : \mathbf{N} \to \mathbf{R}_+ | (\exists c_1, c_2 \in \mathbf{R}_+^*)(\exists n_0 \in \mathbf{N})(\forall n \geq n_0) \\
&\qquad\qquad (c_1 g(n) \leq f(n) \leq c_2 g(n))\} \\
o(g) &= \{f : \mathbf{N} \to \mathbf{R}_+ | (\forall c \in \mathbf{R}_+^*)(\exists n_0 \in \mathbf{N})(\forall n \geq n_0)(f(n) \leq cg(n))\}
\end{aligned}$$

Definiţia 2.7.1.1. Fie f şi g funcţii de la $\mathbf{N}$ la $\mathbf{R}_+$, şi $X \in \{\mathcal{O}, \Omega, \Theta, o\}$. Spunem că *$f$ este de ordinul X al lui g*, şi notăm $f(n) = X(g(n))$, dacă $f \in X(g)$.

Notaţia "$\mathcal{O}$" a fost introdusă de Paul Bachmann în 1894 [4], fiind apoi popularizată intens de Edmund Landau [108, 109], în timp ce notaţia "o" îi este datorată lui Landau [108] [9].

Intuitiv, "$f(n) = \mathcal{O}(g(n))$" înseamnă că f nu creşte mai repede, din punct de vedere asimptotic, decât g (eventual multiplicată printr-o constantă). Atragem atenţia asupra notaţiei "$f(n) = \mathcal{O}(g(n))$"; ea nu trebuie gândită ca o egalitate, ci ca

[9] Toate aceste notaţii pot fi considerate într-un cadru mai general, cel al funcţiilor definite pe $\mathbf{R}$ cu valori în $\mathbf{R}$. Pentru necesităţile noastre, varianta deja considerată este suficientă.

apartenenţa funcţiei f la mulţimea $\mathcal{O}(g)$. Cititorul s-ar putea arăta nedumerit, şi pe bună dreptate, de adoptarea a încă unei notaţii (cea prin "=") atâta timp cât notaţia prin "$\in$" este clară şi corectă din punct de vedere formal. Adoptarea acestei noi notaţii este datorată faptului că aceasta este încetăţenită în rândul matematicienilor şi informaticienilor.

Evident, o notaţie de genul $f(n) \neq \mathcal{O}(g(n)$ înseamnă că f nu este de ordinul $\mathcal{O}$ al lui g.

Următoarea propoziţie, a cărei demonstraţie urmează cu uşurinţă de la definiţii, prezintă câteva din proprietăţile de bază ale ordinelor de mărime.

Propoziţia 2.7.1.1. Fie f, g, h şi k funcţii de la $\mathbf{N}$ la $\mathbf{R}_+$. Atunci, au loc următoarele proprietăţi:

(1) $f(n) = \mathcal{O}(f(n))$;

(2) dacă $f(n) = \mathcal{O}(g(n))$ şi $g(n) = \mathcal{O}(h(n))$, atunci $f(n) = \mathcal{O}(h(n))$;

(3) $f(n) = \mathcal{O}(g(n))$ dacă şi numai dacă $g(n) = \Omega(f(n))$;

(4) $f(n)) = \Theta(g(n))$ dacă şi numai dacă $f(n) = \mathcal{O}(g(n))$ şi $f(n) = \Omega(g(n))$;

(5) dacă $f(n) = \mathcal{O}(h(n))$ şi $g(n) = \mathcal{O}(k(n))$, atunci $(f \cdot g)(n) = \mathcal{O}(h(n)k(n))$ şi $(f+g)(n) = \mathcal{O}(max\{h(n), k(n)\})$;

(6) dacă există $n_0 \in \mathbf{N}$ astfel încât $g(n) \neq 0$ pentru orice $n \geq n_0$, atunci $f(n) = o(g(n))$ dacă şi numai dacă $lim_{n\to\infty} \dfrac{f(n)}{g(n)} = 0$.

Următoarele inegalităţi sunt foarte utile în stabilirea ordinelor de mărime pentru diverse funcţii:

- (Formula lui Stirling)

$$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{\frac{1}{12n+1}} \leq n! \leq \sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{\frac{1}{12n}},$$

  pentru orice $n \geq 1$;

- pentru orice constante reale ϵ şi c astfel încât $0 < \epsilon < 1 < c$, are loc

$$1 < ln\,ln\,n < ln\,n < e^{\sqrt{(ln\,n)(ln\,ln\,n)}} < n^{\epsilon} < n^{c} < n^{ln\,n} < c^{n} < n^{n} < c^{c^n}$$

  (fiecare inegalitate este satisfăcută pentru orice $n \geq n_0$, unde n_0 este ales convenabil pentru fiecare inegalitate).

Astfel putem obţine:

- dacă f este un polinom de grad k cu coeficienţi reali, dar astfel încât coeficientul termenului de grad maxim este pozitiv, iar funcţia polinomială asociată (notată tot prin f) ia valori reale pozitive, atunci $f(n) = \Theta(n^k)$;

- pentru orice constantă reală $c > 1$, $log_c n = \Theta(log\, n)$ (log reprezintă funcţia logaritm în baza 2);
- pentru orice număr real ϵ cu $0 < \epsilon < 1$, $log\, n = \mathcal{O}(n^\epsilon)$;
- pentru orice număr natural $k \geq 1$, $log^k n = \mathcal{O}(n)$;
- $n! = \Omega(2^n)$ şi $n! = o(n^n)$ [10];
- $log(n!) = \Theta(nlog\, n)$;
- dacă $f : \mathbf{N} \to \mathbf{R}_+$ este o funcţie astfel încât există $n_0 \in \mathbf{N}$ cu proprietatea $f(n) \geq 1$, pentru orice $n \geq n_0$, atunci

$$\frac{1}{2}2^{\lceil log_2 f(n) \rceil} \leq f(n) \leq 2^{\lceil log_2 f(n) \rceil},$$

pentru orice $n \geq n_0$, ceea ce conduce la $f(n) = \Theta(2^{\lceil log_2 f(n) \rceil})$.

Atragem însă atenţia asupra unor situaţii de genul $4^n \neq \mathcal{O}(2^n)$ [11].

Dacă $\mathcal{A}$ şi $\mathcal{B}$ sunt mulţimi de funcţii ca cele definite mai sus ($\mathcal{O}(g)$ etc.), iar f este o funcţie de la $\mathbf{N}$ la $\mathbf{R}_+$, atunci vom nota:

- $f + \mathcal{A} = \{f + g | g \in \mathcal{A}\}$;
- $\mathcal{A} + \mathcal{B} = \{f + g | f \in \mathcal{A}, g \in \mathcal{B}\}$;
- $f\mathcal{A} = \{f \cdot g | g \in \mathcal{A}\}$. Dacă f este funcţia constantă c, atunci vom scrie $c\mathcal{A}$ în loc de $f\mathcal{A}$;
- $\mathcal{AB} = \{fg | f \in \mathcal{A}, g \in \mathcal{B}\}$;
- $\mathcal{O}(\mathcal{A}) = \bigcup_{f \in \mathcal{A}} \mathcal{O}(f)$.

Egalitatea $\mathcal{A} = \mathcal{B}$ va fi înţeleasă prin incluziune (pentru orice funcţie f, dacă $f \in \mathcal{A}$, atunci $f \in \mathcal{B}$). Această ultimă convenţie este în strânsă legătură cu convenţia deja adoptată ($f(n) = \mathcal{O}(g(n))$, de exemplu). Astfel, $\mathcal{O}(f(n)) = \mathcal{O}(g(n))$ ne spune că orice funcţie ce este de ordinul $\mathcal{O}$ al lui f este şi de ordinul $\mathcal{O}$ al lui g (dar nu în mod necesar şi invers), iar $f(n) + \mathcal{O}(g(n)) = \mathcal{O}(h(n))$ ne spune că suma dintre f şi o funcţie de ordinul $\mathcal{O}$ al lui g este o funcţie de ordinul $\mathcal{O}$ al lui h. De exemplu,

$$\frac{1}{3}n^3 + \mathcal{O}(n^2) = \mathcal{O}(n^3).$$

[10] De fapt, observăm că $lim_{n\to\infty} \frac{n!}{(\frac{n}{e})^n \sqrt{2\pi n}} = 1$, ceea ce ne arată că $(\frac{n}{e})^n \sqrt{2\pi n}$ constituie o şi mai bună aproximare asimptotică pentru $n!$. Astfel, este interesant de remarcat că, pentru $n = 100$, $e^{\frac{1}{1201}} \approx 1.00083299$ şi $e^{\frac{1}{1200}} \approx 1.00083368$. Ca urmare, formula lui Stirling oferă margini superioare şi inferioare foarte apropiate pentru aproximarea lui $n!$ (pentru $n \geq 100$ diferenţa dintre acestea este mai mică decât 10^{-6}).

[11] Dacă presupunem, prin contradicţie, că $4^n = \mathcal{O}(2^n)$, atunci există o constantă reală $c > 0$ şi un număr natural n_0 astfel încât $4^n \leq c2^n$, pentru orice $n \geq n_0$. De aici urmează $2^n \leq c$, pentru orice $n \geq n_0$, ceea ce este fals.

Putem accepta şi notaţii de genul $f(n) = g(n) + \mathcal{O}(h(n))$ pentru a specifica faptul că f este suma dintre g şi o funcţie de ordinul $\mathcal{O}$ al lui h (sau, altfel spus, f este un element al mulţimii $g + \mathcal{O}(h)$).

Următoarea propoziţie urmează cu uşurinţă de la definiţii (dar atragem încă o dată atenţia asupra faptului că egalitatea dintre mulţimile noastre de funcţii desemnează de fapt incluziune de la stânga la dreapta).

Propoziţia 2.7.1.2. Fie f şi g funcţii de la $\mathbf{N}$ la $\mathbf{R}_+$ şi $c \in \mathbf{R}_+$. Atunci au loc următoarele proprietăţi:

(1) $\mathcal{O}(f(n)) + \mathcal{O}(g(n)) = \mathcal{O}(f(n) + g(n))$;

(2) $c\mathcal{O}(f(n)) = \mathcal{O}(f(n))$;

(3) $\mathcal{O}(\mathcal{O}(f(n))) = \mathcal{O}(f(n))$;

(4) $\mathcal{O}(f(n))\mathcal{O}(g(n)) = \mathcal{O}(f(n)g(n))$;

(5) $\mathcal{O}(f(n)g(n)) = f(n)\mathcal{O}(g(n))$.

2.7.2. Timpul de execuţie al unui algoritm

Estimarea "timpului" necesar execuţiei unui algoritm (calculării unei funcţii) se realizează în raport cu o anumită unitate de măsură a datelor de intrare. Cel mai adesea se ia în considerare "lungimea" reprezentării datelor de intrare (a operanzilor) într-o bază b [12] [13]. Menţionăm întâi că orice număr natural n poate fi *reprezentat în baza* $b \geq 2$ în forma

$$n = n_{k-1}b^{k-1} + \cdots + n_0,$$

unde $0 \leq n_i < b$ pentru orice $0 \leq i < k$, şi $n_{k-1} \neq 0$. Această reprezentare este complet determinată de secvenţa de numere

$$(n_{k-1}, \ldots, n_0)_b,$$

motiv pentru care această secvenţă va fi numită *reprezentarea în baza b a lui n*. Numărul $k > 0$ este numit *lungimea reprezentării* lui n în baza b (sau *lungimea* lui n, atunci când baza b este subînţeleasă din context), n_{k-1} se numeşte *cifra cea mai semnificativă*, iar n_0 *cifra cea mai puţin semnificativă* a reprezentării lui n în baza b.

[12] O *bază de numeraţie* este un număr natural $b \geq 2$. Numerele i, cu $0 \leq i < b$, sunt numite *cifrele bazei b*.

[13] Reprezentarea internă a datelor în calculator se face utilizând codificarea binară a acestora. Operaţiile cu date sunt astfel convertite în operaţii cu şiruri binare (deplasare la stânga sau la dreapta cu o poziţie, adunare de şiruri binare etc.). Ca urmare, complexitatea executării de către calculator a unor operaţii cu anumite tipuri de date se rezumă la complexitatea realizării unor operaţii cu şiruri binare, aceasta din urmă fiind "măsurată" în raport cu lungimea şirurilor.

Atunci când $k = 1$ spunem că n este *număr în precizie simplă*; altfel, n este un *număr în precizie multiplă*.

Relaţia dintre n şi k este dată prin

$$k = \lfloor \log_b n \rfloor + 1$$

(cu convenţia $\log_b 0 = 0$). Ca urmare, funcţia $f(n)$ ce furnizează lungimea reprezentării în baza b a lui n satisface relaţia $f(n) = \Theta(\log_b n)$.

Atunci când $b \leq 10$, notaţia în baza b va fi simplificată la $(n_{k-1} \cdots n_0)_b$. Putem face această simplificare şi pentru $10 < b \leq 35$ dacă folosim, de exemplu, alfabetul englez şi renotăm numerele 10, 11 etc. prin A, B etc., în această ordine. Astfel, $(1A2)_{16}$ constituie aceeaşi reprezentare în baza 16 ca şi secvenţa $(1, 10, 2)_{16}$. Atunci când vom lucra cu baza 2 sau 10 vom folosi o nouă simplificare, obţinută prin eliminarea parantezelor şi a indicelui bazei. Cifrele utilizate pentru scrierea unui număr în baza 2 sunt numite *cifre binare* sau *biţi*. *Secvenţele binare* sunt secvenţe de biţi. Uneori este convenabil să completăm la stânga cu zerouri reprezentarea în baza b a lui n. Ne vom referi la şirurile obţinute ca fiind tot reprezentarea în baza b a lui n (această convenţie are caracter pur tehnic).

Presupunem că cititorul este familiarizat cu operaţiile uzuale de adunare, scădere, înmulţire şi împărţire cu numere scrise într-o bază b. Acestea se realizează prin repetarea de un număr finit de ori a următoarelor operaţii considerate primitive:

- compararea a două cifre ale bazei;
- adunare, scădere şi înmulţire a 2 cifre ale bazei, luând în considerare şi transportul, având drept răspuns o cifră a bazei şi un transport;
- împărţire a unui număr format din două cifre ale bazei la o cifră a bazei, având drept răspuns un cât şi un rest (ambele fiind cifre ale bazei).

Regulile de bază folosite în evaluarea complexităţii unui algoritm sunt:

- complexitatea unei structuri repetitive (de tip *for*, *while*, *until*) este dată de complexitatea testului la care se adaugă complexitatea maximă a corpului structurii, înmulţind totul cu numărul de repetări ale acestuia;
- complexitatea unei structuri secvenţiale este complexitatea maximă a componentelor structurii;
- complexitatea structurii *if-then-else* (*if-then*) este dată de complexitatea testului la care se adaugă complexitatea maximă a ramurilor (a ramurii).

Vom trece acum în revistă complexitatea realizării operaţiilor de bază, cum ar fi adunarea, scăderea, înmulţirea etc. Vom urma cu precădere [201], unde cititorul poate găsi detalii complete asupra algoritmilor discutaţi sumar mai jos şi a complexităţii acestora.

Adunarea şi scăderea a două numere cu reprezentare binară pe cel mult k biţi pot fi realizate în complexitate timp $\mathcal{O}(k)$.

Înmulţirea realizată în variantă "şcolărească" necesită complexitate timp $\mathcal{O}(k^2)$. Dacă însă se utilizează algoritmul Karatsuba, complexitatea timp devine $\mathcal{O}(k^{\log 3})$, iar dacă se utilizează transformata Fourier discretă, complexitatea scade dramatic la $\mathcal{O}(k \log k)$. Cum însă înmulţirea realizată prin transformata Fourier discretă este eficientă doar pentru valori mari ale lui k (de obicei, cel puţin 1000), în practică, algoritmul Karatsuba se dovedeşte de preferat.

Împărţirea realizată prin algoritmul uzual de împărţire necesită complexitate timp $\mathcal{O}(k^2)$. Ea poate fi realizată mult mai eficient utilizând un rezultat care spune că, la un pas de împărţire, o cifră a câtului se poate determina prin cel mult două încercări. Astfel, determinarea câtului q şi a restului r al împărţirii lui a la b se poate face în complexitate timp $\mathcal{O}((\log b)(\log q))$. Există şi o variantă recursivă a împărţirii, bazată oarecum pe ideea ce stă la baza algoritmului Karatsuba, ce necesită $\mathcal{O}(k^{\log 3} + k \log k)$.

O analiză simplistă a algoritmului lui Euclid ne arată că acesta are complexitatea $\mathcal{O}(k^3)$. Însă o analiză atentă conduce la un rezultat mult mai bun. Împărţirea lui r_i la r_{i+1} cu obţinerea câtului q_{i+2} şi a restului r_{i+2}, utilizând notaţiile din Secţiunea 2.2, se poate realiza în $\mathcal{O}((\log r_{i+1})(\log q_{i+2}))$. Atunci

$$\begin{aligned} \textstyle\sum_{i=-1}^{n-1} (\log r_{i+1})(\log q_{i+2}) &\leq (\log b) \textstyle\sum_{i=-1}^{n-1} \log q_{i+2} \\ &= (\log b)(\log q_1 \cdots q_{n+1}) \end{aligned}$$

Nu este greu de văzut că

$$q_1 \cdots q_{n+1} \leq a,$$

ceea ce conduce la complexitatea timp $\mathcal{O}((\log a)(\log b))$. Deci dacă numerele se reprezintă pe cel mult k biţi, atunci complexitatea algoritmului lui Euclid este $\mathcal{O}(k^2)$. Există şi alţi algoritmi de calcul al celui mai mare divizor comun a două numere. Cea mai eficientă soluţie este de complexitate $\mathcal{O}(k^2/\log k)$.

Exponenţierea modulară, adică calculul lui $a^n \ mod \ m$, necesită complexitate timp $\mathcal{O}(k^3)$, presupunând că a, n şi m se reprezintă pe cel mult k biţi. Există multe soluţii pentru această problemă, depinzând de diverse particularităţi ale exponentului sau modulului, soluţii de complexitate mult mai bună decât cea menţionată mai sus. Toate acestea pot fi găsite în [201].

Atunci când pentru o problemă nu se cunoaşte nici un algoritm de complexitate timp polinomială, iar algoritmii existenţi sunt impracticabili pentru valori rezonabile ale datelor de intrare, vom spune că problema este *dificilă* sau *grea* sau *intractabilă*. De exemplu, factorizarea numerelor este o problemă dificilă. Pentru a înţelege ce înseamnă aceasta, aducem la cunoştinţa cititorului următorul rezultat. Pe data de 9 mai 2005, o echipă a Agenţiei Federale Germane pentru securitatea informaţiei, compusă din F. Bahr, M. Boehm, J. Franke şi T. Kleinjung, a anunţat factorizarea unui număr de 200 de cifre, număr cunoscut sub denumirea de RSA-200. Acest număr face parte dintr-o selecţie de numere propuse de compania americană *RSA Security*, numere ce au exact 2 factori primi şi care sunt considerate ca fiind dificil de factorizat ("pietre de încercare" pentru problema factorizării). Echipa germană a utilizat pentru factorizarea acestui număr o reţea de calculatoare ce au lucrat în paralel.

Timpul CPU necesar factorizării acestui număr folosind un procesor AMD Opteron la 2.2 GHz ar fi fost de aproximativ 55 de ani (a se vedea rubrica "challenges" la `http://www.rsasecurity.com/rsalabs//wiki/RSA-200`). Oricum, echipa germană a început lucrul la sfârşitul anului 2003 şi factorizarea s-a încheiat cu succes în mai 2005.